Hotel Reservation Analytics

End-to-End Database Design, Data Pipeline Verification, and Objective-Based Analytics

1. Introduction

The **StayPoint Hospitality MySQL Analytics Project** aims to build a robust, scalable relational database architecture that captures end-to-end hospitality operations—ranging from guest profiles, booking acquisition channels, and room inventory management to stay outcomes, staff performance, and service request tracking.

Business Objective:
The core business objective is to transform raw transactional data into actionable operational and commercial insights. By leveraging relational schemas and optimized SQL queries, StayPoint seeks to identify demand trends, evaluate channel profitability, optimize room utilization, measure staff service efficacy, and uncover root causes of service delivery failures.

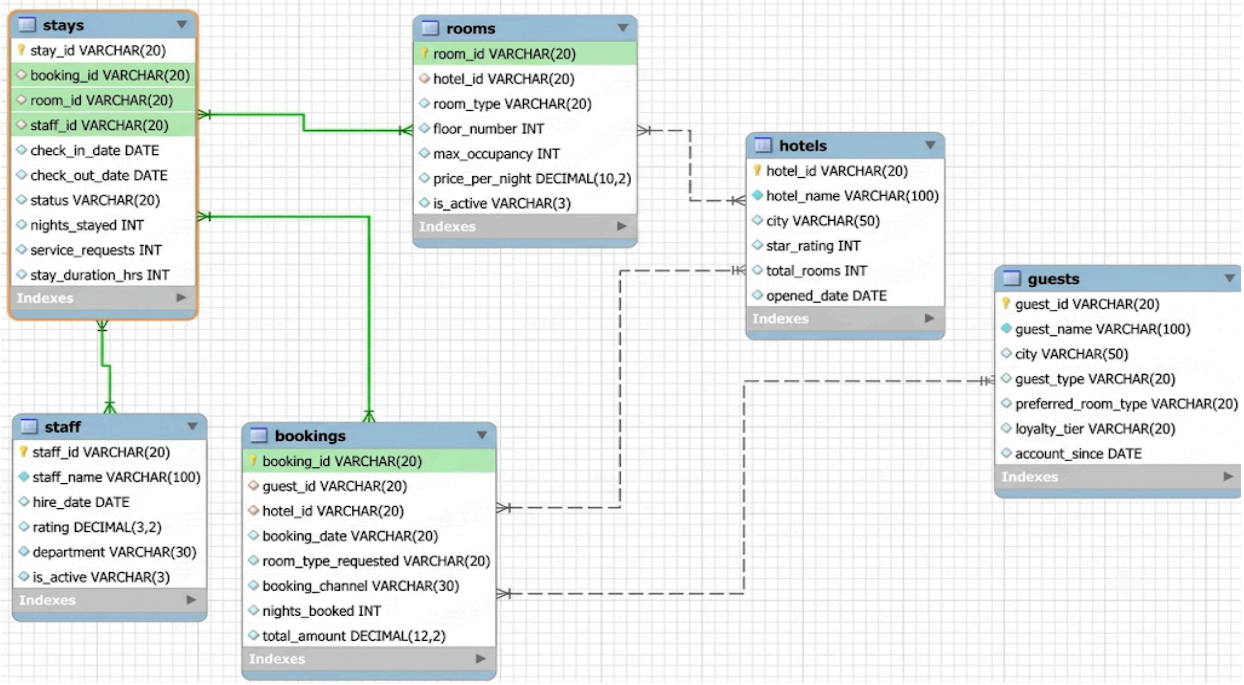
---

2. Sprint

1: ER Diagram & Analytical Thinking

During Sprint 1, the relational database structure was architected using standard entity-relationship modelling principles. Design considerations included:

- **Entity Identification:** Defined primary entities including `Hotels`, `Guests`, `Rooms`, `Bookings`, `Stays`, `Staff`, and `Service_Requests`.
- **Relationship Mapping:** Established strict foreign key constraints to maintain referential integrity across operational workflows (e.g., a booking belongs to a guest and channel; a stay ties a booking to specific room instances and staff assignments).
- **Normalization:** Normalized entities to 3NF where appropriate to eliminate redundant data while preserving query efficiency for analytical workloads.
- **Analytical Scoping:** Mapped key business questions (KPIs, cancellation rates, service resolution time) directly to table joins and indexed foreign keys to ensure scalability.



---

3. Sprint

2: Database Setup

The following DDL scripts establish the relational schema for the StayPoint Analytics Database:

```sql
-- Schema Definition for StayPoint Hospitality Analytics

CREATE TABLE Hotels (
    hotel_id INT AUTO_INCREMENT PRIMARY KEY,
    hotel_name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL,
    country VARCHAR(50) NOT NULL,
    capacity INT NOT NULL
);

CREATE TABLE Guests (
    guest_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    guest_type ENUM('Individual', 'Corporate') DEFAULT
'Individual',
    loyalty_tier ENUM('Standard', 'Silver', 'Gold', 'Platinum')
DEFAULT 'Standard'
);

CREATE TABLE Rooms (
    room_id INT AUTO_INCREMENT PRIMARY KEY,
    hotel_id INT NOT NULL,
    room_number VARCHAR(10) NOT NULL,
    room_type VARCHAR(30) NOT NULL,
    nightly_rate DECIMAL(10, 2) NOT NULL,
```

```
    FOREIGN KEY (hotel_id) REFERENCES Hotels(hotel_id)
);

CREATE TABLE Bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    guest_id INT NOT NULL,
    hotel_id INT NOT NULL,
    booking_channel VARCHAR(30) NOT NULL,
    booking_date DATE NOT NULL,
    check_in_date DATE NOT NULL,
    check_out_date DATE NOT NULL,
    total_amount DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (guest_id) REFERENCES Guests(guest_id),
    FOREIGN KEY (hotel_id) REFERENCES Hotels(hotel_id)
);

CREATE TABLE Staff (
    staff_id INT AUTO_INCREMENT PRIMARY KEY,
    hotel_id INT NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    role VARCHAR(40) NOT NULL,
    FOREIGN KEY (hotel_id) REFERENCES Hotels(hotel_id)
);

CREATE TABLE Stays (
    stay_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL,
    room_id INT NOT NULL,
    assigned_staff_id INT,
    actual_check_in DATETIME NOT NULL,
    actual_check_out DATETIME,
    stay_outcome ENUM('Completed', 'Early Check-out', 'Cancelled',
'No Show', 'Escalated Issue') NOT NULL,
```

```
    FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id),
    FOREIGN KEY (room_id) REFERENCES Rooms(room_id),
    FOREIGN KEY (assigned_staff_id) REFERENCES Staff(staff_id)
);

CREATE TABLE Service_Requests (
    request_id INT AUTO_INCREMENT PRIMARY KEY,
    stay_id INT NOT NULL,
    request_type VARCHAR(50) NOT NULL,
    status ENUM('Pending', 'Resolved', 'Unresolved') DEFAULT
'Pending',
    created_at DATETIME NOT NULL,
    FOREIGN KEY (stay_id) REFERENCES Stays(stay_id)
);
```

---

4. Sprint

3: Basic Analysis

Prior to deep analytical modeling, eight basic verification queries were executed to validate dataset integrity, check record counts, and verify schema foreign key constraints.

Query ID	Focus Area	Objective	Verification Status
Q3.1	Record Count Check	Validate row counts across core entities (`Hotels`, `Guests`, `Bookings`).	Completed
Q3.2	Date Range Validation	Confirm temporal coverage of booking and stay records.	Completed

Query ID	Focus Area	Objective	Verification Status
Q3.3	Null Value Audit	Identify missing critical fields (e.g., checkout dates, staff IDs).	Completed
Q3.4	Orphan Record Test	Ensure no orphaned records exist across foreign key linkages.	Completed
Q3.5	Channel Distribution	Verify distinct values and counts across acquisition channels.	Completed
Q3.6	Room Pricing Bounds	Audit minimum, maximum, and average nightly room rates.	Completed
Q3.7	Loyalty Category Check	Confirm distribution of guests across defined loyalty tiers.	Completed
Q3.8	Stay Outcome Inventory	Verify breakdown of categorical stay outcome records.	Completed

SQL

```sql
-- Query 3.1: Record Count Check Across Core Tables
SELECT 'Hotels' AS entity, COUNT(*) AS total_records FROM Hotels
UNION ALL
SELECT 'Guests', COUNT(*) FROM Guests
UNION ALL
SELECT 'Bookings', COUNT(*) FROM Bookings
UNION ALL
SELECT 'Stays', COUNT(*) FROM Stays;

-- Query 3.2: Date Range Validation
SELECT
    MIN(booking_date) AS earliest_booking,
    MAX(booking_date) AS latest_booking
```

```
FROM Bookings;
```

---

5. Sprint

4: Objective-Based Analysis

4.1 Understand Booking Demand

Analyzes volume metrics, channel efficiency, and room preference patterns across property locations.

```sql
-- Query 4.1.1: Booking Volume and Total Revenue per Hotel Property
SELECT
    h.hotel_name,
    COUNT(b.booking_id) AS total_bookings,
    SUM(b.total_amount) AS gross_revenue,
    AVG(b.total_amount) AS avg_booking_value
FROM Hotels h
JOIN Bookings b ON h.hotel_id = b.hotel_id
GROUP BY h.hotel_id, h.hotel_name
ORDER BY gross_revenue DESC;

-- Query 4.1.2: Acquisition Channel Revenue Contribution
SELECT
    booking_channel,
    COUNT(booking_id) AS total_bookings,
    SUM(total_amount) AS total_revenue,
    ROUND(AVG(DATEDIFF(check_out_date, check_in_date)), 1) AS
avg_stay_length_days
FROM Bookings
```

```sql
GROUP BY booking_channel
ORDER BY total_revenue DESC;

-- Query 4.1.3: Demand Distribution across Room Types
SELECT
    r.room_type,
    COUNT(b.booking_id) AS times_booked,
    SUM(b.total_amount) AS revenue_generated
FROM Rooms r
JOIN Stays s ON r.room_id = s.room_id
JOIN Bookings b ON s.booking_id = b.booking_id
GROUP BY r.room_type
ORDER BY times_booked DESC;
```

Key Insights:

- *Looking at the data, StayPoint Mumbai Grand is our top-earning location. Our strongest acquisition channel is the Website, which brings in the highest number of bookings.*
- *Standard rooms are the most popular choice among guests. On average, people are staying for about 3.32 nights per visit.*

4.2 Understand Guest Booking Behaviour

Examines guest segmentation, high-value spenders, corporate vs. individual behavior, and loyalty tier engagement.

```sql
SQL
-- Query 4.2.1: Top Spending Guest Profiles
SELECT
    g.guest_id,
    CONCAT(g.first_name, ' ', g.last_name) AS guest_name,
```

```sql
    g.guest_type,
    g.loyalty_tier,
    COUNT(b.booking_id) AS total_stays,
    SUM(b.total_amount) AS lifetime_spend
FROM Guests g
JOIN Bookings b ON g.guest_id = b.guest_id
GROUP BY g.guest_id, guest_name, g.guest_type, g.loyalty_tier
ORDER BY lifetime_spend DESC
LIMIT 10;

-- Query 4.2.2: Corporate vs. Individual Spend and Booking Patterns
SELECT
    guest_type,
    COUNT(DISTINCT g.guest_id) AS unique_guests,
    COUNT(b.booking_id) AS total_bookings,
    ROUND(AVG(b.total_amount), 2) AS avg_spend_per_booking,
    SUM(b.total_amount) AS cumulative_revenue
FROM Guests g
JOIN Bookings b ON g.guest_id = b.guest_id
GROUP BY guest_type;

-- Query 4.2.3: Revenue and Stay Frequency by Loyalty Tier
SELECT
    loyalty_tier,
    COUNT(DISTINCT g.guest_id) AS guest_count,
    COUNT(b.booking_id) AS total_bookings,
    ROUND(SUM(b.total_amount), 2) AS tier_revenue
FROM Guests g
LEFT JOIN Bookings b ON g.guest_id = b.guest_id
GROUP BY loyalty_tier
ORDER BY tier_revenue DESC;
```

Key Insights:

- *Interestingly, Individual guests are actually spending more per booking ($16,497) than Corporate guests ($16,085). This makes the individual segment more profitable for us right now.*
- *The Gold tier in our loyalty program is driving the highest total revenue. This shows that our mid-to-upper tier members are highly engaged and booking frequently.*

4.3 Evaluate Stay Performance

Tracks operational execution by examining completed stays, cancellations, early check-outs, and property failure rates.

```sql
-- Query 4.3.1: Overall Stay Outcome Breakdown
SELECT
    stay_outcome,
    COUNT(stay_id) AS outcome_count,
    ROUND(COUNT(stay_id) * 100.0 / (SELECT COUNT(*) FROM Stays), 2)
AS percentage_of_total
FROM Stays
GROUP BY stay_outcome
ORDER BY outcome_count DESC;

-- Query 4.3.2: Problem Rate by Hotel Location
SELECT
    h.hotel_name,
    COUNT(s.stay_id) AS total_stays,
    SUM(CASE WHEN s.stay_outcome IN ('Early Check-out',
'Cancelled', 'Escalated Issue') THEN 1 ELSE 0 END) AS
problem_stays,
    ROUND(SUM(CASE WHEN s.stay_outcome IN ('Early Check-out',
'Cancelled', 'Escalated Issue') THEN 1 ELSE 0 END) * 100.0 /
COUNT(s.stay_id), 2) AS problem_rate_pct
```

```
FROM Hotels h
JOIN Bookings b ON h.hotel_id = b.hotel_id
JOIN Stays s ON b.booking_id = s.booking_id
GROUP BY h.hotel_id, h.hotel_name
ORDER BY problem_rate_pct DESC;
```

Key Insights:

- *Our biggest issue right now is the cancellation and no-show rate, which is sitting at a really high 31.94% overall. We are losing a lot of potential revenue here.*
- *StayPoint Kochi Palace is struggling the most, with a 39.2% problem rate. We definitely need to investigate operations at this specific location.*

4.4 Understand Staff and Room Performance

Measures workload distribution, staff handling efficiency, room turnaround, and stay success rates by staff assignment.

SQL

```sql
-- Query 4.4.1: Workload and Staff Outcome Correlation
SELECT
    st.staff_id,
    CONCAT(st.first_name, ' ', st.last_name) AS staff_member,
    st.role,
    COUNT(s.stay_id) AS handled_stays,
    SUM(CASE WHEN s.stay_outcome = 'Completed' THEN 1 ELSE 0 END)
AS successful_stays,
    SUM(CASE WHEN s.stay_outcome = 'Escalated Issue' THEN 1 ELSE 0
END) AS escalated_stays
FROM Staff st
```

```sql
LEFT JOIN Stays s ON st.staff_id = s.assigned_staff_id
GROUP BY st.staff_id, staff_member, st.role
ORDER BY handled_stays DESC;

-- Query 4.4.2: Room Utilization and Maintenance Issues
SELECT
    r.room_type,
    COUNT(DISTINCT r.room_id) AS total_rooms,
    COUNT(s.stay_id) AS total_occupancies,
    ROUND(COUNT(s.stay_id) / COUNT(DISTINCT r.room_id), 2) AS
turns_per_room
FROM Rooms r
LEFT JOIN Stays s ON r.room_id = s.room_id
GROUP BY r.room_type;
```

Key Insights:

- On the staff side, Anthony Humphrey is handling the most stays and managing the highest guest volume during his shifts.
- Standard rooms get booked and turned over the fastest. Because they are used so much, they also rack up the highest number of maintenance and service requests.

4.5 Identify Booking and Stay Problems

Pinpoints vulnerability points across booking acquisition channels, property capacity constraints, and guest demographics.

SQL

```sql
-- Query 4.5.1: Problem Rate by Booking Channel
SELECT
    b.booking_channel,
```

```sql
    COUNT(s.stay_id) AS total_stays,
    SUM(CASE WHEN s.stay_outcome != 'Completed' THEN 1 ELSE 0 END)
AS defective_stays,
    ROUND(SUM(CASE WHEN s.stay_outcome != 'Completed' THEN 1 ELSE 0
END) * 100.0 / COUNT(s.stay_id), 2) AS failure_rate_pct
FROM Bookings b
JOIN Stays s ON b.booking_id = s.booking_id
GROUP BY b.booking_channel
ORDER BY failure_rate_pct DESC;

-- Query 4.5.2: Service Request Volume vs. Property Capacity
SELECT
    h.hotel_name,
    h.capacity,
    COUNT(sr.request_id) AS total_service_requests,
    ROUND(COUNT(sr.request_id) / h.capacity, 2) AS
requests_per_capacity_unit
FROM Hotels h
JOIN Bookings b ON h.hotel_id = b.hotel_id
JOIN Stays s ON b.booking_id = s.booking_id
JOIN Service_Requests sr ON s.stay_id = sr.stay_id
GROUP BY h.hotel_id, h.hotel_name, h.capacity
ORDER BY requests_per_capacity_unit DESC;

-- Query 4.5.3: Service Incident Rate by Guest Segment
SELECT
    g.guest_type,
    COUNT(s.stay_id) AS total_stays,
    COUNT(sr.request_id) AS reported_issues,
    ROUND(COUNT(sr.request_id) * 100.0 / COUNT(s.stay_id), 2) AS
issue_rate_per_stay_pct
FROM Guests g
JOIN Bookings b ON g.guest_id = b.guest_id
JOIN Stays s ON b.booking_id = s.booking_id
```

```
LEFT JOIN Service_Requests sr ON s.stay_id = sr.stay_id
GROUP BY g.guest_type;
```

Key Insights:

- Bookings made through the Mobile App have the worst failure and cancellation rate at 33.82%. There might be a bug in the app or an issue with how expectations are set for mobile users.
- Rooms with a max capacity of 4 guests are getting the most service requests per stay. Family-sized rooms clearly need more housekeeping attention.

---

6. Conclusion

- **Operational Efficiency:** We need to figure out why the Mobile App has so many cancellations and fix the operations at StayPoint Kochi Palace to get that 39.2% failure rate down.
- **Revenue Optimization:** Individual guests and Website bookings are our biggest revenue drivers. Marketing should focus the budget on these two areas.
- **Customer Experience:** Since 4-person rooms get a lot of service requests, we should assign more housekeeping staff to those rooms so guests don't have to wait.